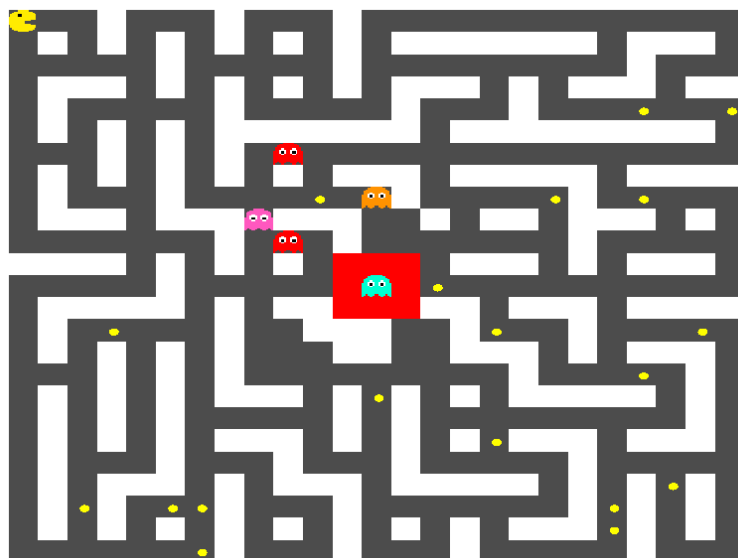


Université Marie & Louis Pasteur
UFR Sciences et Techniques

Projet L3 : Pacman en réseau

Rapport de Projet

Licence CMI Informatique - 3ème année
2025-2026



Thomas COUTANT
Benoît LITAMPHA
Auriane PETER

Encadrant : Julien BERNARD

Remerciements

Table des matières

Remerciements	1
Introduction	5
1 Présentation du projet	6
1.1 Contexte académique	6
1.2 Présentation générale du jeu Pac-Man	6
1.3 Objectifs généraux du projet	7
2 Définition du sujet et problématique	9
2.1 Problématique du jeu en réseau	9
2.2 Choix techniques	9
2.3 Répartition des rôles	10
3 Architecture et conception	11
3.1 Architecture générale	11
3.2 Structure du serveur	11
3.3 Structure du client	12
3.4 Protocoles et échanges réseau	12
4 Réalisation et déroulement du projet	14
4.1 Mise en place de l'environnement de développement	14
4.2 Conception de l'architecture du serveur	15
4.2.1 Mise en place de la structure	15
4.2.2 Mise en place du jeu	17
5 Bilan et perspectives	23
5.1 Compétences acquises	23
5.2 Résultat final	23
5.3 Améliorations possibles	23
Conclusion	24
Sitographie	25
Résumé	26
Mots-Clés	26
Abstract	26

Key-Words

26

Table des figures

1.1	PacMan : le jeu original	7
2.1	Model de Chaine de responsabilité	10
3.1	Diagrammes de classe de l'architecture réseau	11
3.2	Diagramme de classes de l'architecture de jeu	12
4.1	Premier prototype	15
4.2	Diagramme de Chaine de responsabilité	16

Introduction

Chapitre 1

Présentation du projet

1.1 Contexte académique

Ce projet a été réalisé dans le cadre du semestre six de la licence informatique de l'Université Marie & Louis Pasteur. Il s'inscrit dans un enseignement visant à mettre en pratique les connaissances acquises au cours de la formation, notamment à travers la réalisation d'un projet informatique de taille conséquente à plusieurs.

Le travail a été mené en groupe de trois étudiants sur une période s'étendant de la mi-octobre à la fin du mois de mars, sous l'encadrement de Julien Bernard, maître de conférences à l'Université Marie & Louis Pasteur. Le projet porte sur le développement d'un jeu vidéo en réseau, de sa conception jusqu'à son implémentation.

Les objectifs pédagogiques de ce projet sont multiples. Il vise en particulier à consolider les compétences en programmation C++, à approfondir la découverte du développement de jeux vidéo, ainsi qu'à mettre en œuvre une architecture réseau de type client-serveur. Le projet permet également de développer des compétences transversales telles que le travail en équipe et la gestion d'un projet informatique sur une durée prolongée.

1.2 Présentation générale du jeu Pac-Man

Le jeu **Pac-Man**, apparu au début des années 1980, est un jeu d'arcade dans lequel le joueur contrôle un personnage évoluant dans un labyrinthe. L'objectif principal est de parcourir l'ensemble du labyrinthe afin de consommer toutes les **pac-gommes** qui s'y trouvent, tout en évitant d'entrer en contact avec des fantômes. Ces derniers constituent les adversaires du joueur et cherchent à bloquer ou intercepter ses déplacements. Certaines **pac-gommes** spéciales permettent temporairement à **Pac-Man** d'inverser les rôles en devenant capable de manger les fantômes, ajoutant ainsi une dimension stratégique au gameplay.

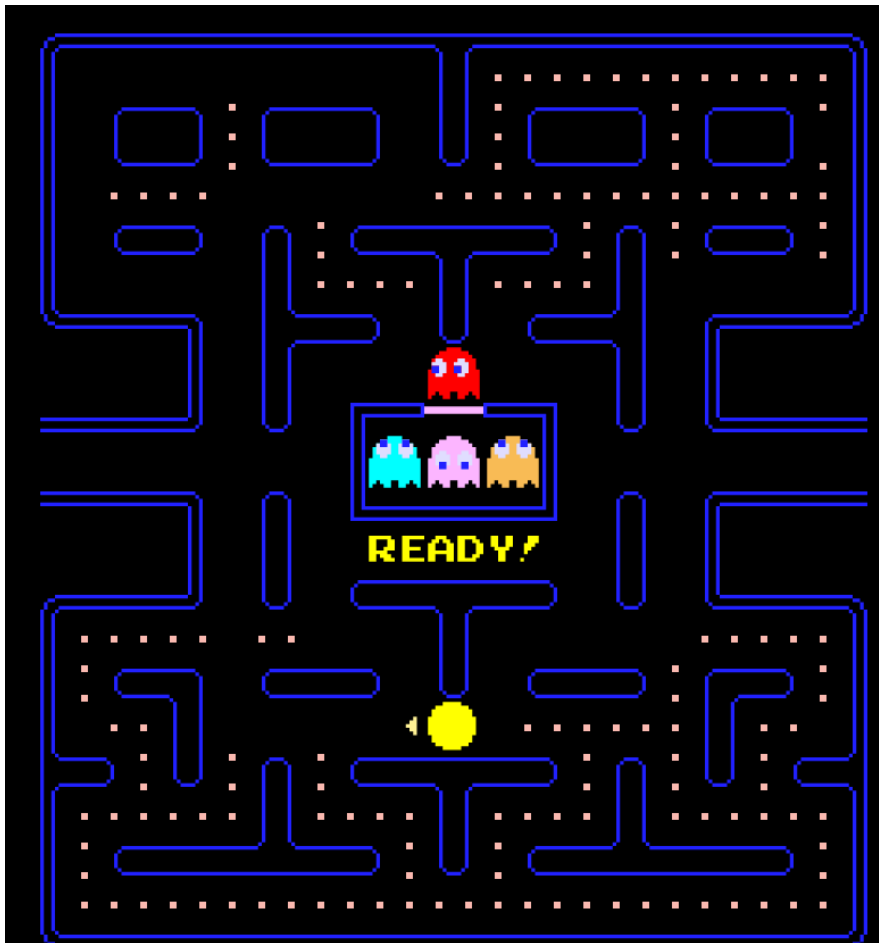


FIGURE 1.1 – PacMan : le jeu original

Dans le cadre de ce projet, le jeu a été adapté afin d'introduire une dimension multijoueur. Une partie oppose un joueur incarnant **Pac-Man** à plusieurs joueurs incarnant des fantômes. Lorsque le nombre de joueurs humains est insuffisant, certains fantômes peuvent être contrôlés par des intelligences artificielles afin de garantir un déroulement cohérent de la partie. Cette approche permet de conserver un équilibre de gameplay, tout en mettant en avant des interactions entre les joueurs.

Le choix de **Pac-Man** comme base pour une adaptation en réseau s'explique par plusieurs raisons. Tout d'abord, ses règles simples et très connues, ce qui permet une prise en main rapide par les joueurs, cela facilite les phases de test et de démonstration. De plus, le gameplay se prête naturellement à une séparation des rôles entre plusieurs joueurs connectés.

1.3 Objectifs généraux du projet

Les objectifs du projet peuvent être divisés en deux catégories principales : les objectifs fonctionnels, liés à l'expérience de jeu, et les objectifs techniques, liés à la conception et à l'implémentation du système.

Du point de vue fonctionnel, l'objectif principal était de concevoir un jeu Pac-Man jouable en réseau, permettant à plusieurs joueurs de participer simultanément à une même partie.

Sur le plan technique, le projet visait à mettre en œuvre une architecture réseau reposant sur un modèle client-serveur. Le serveur devait être capable de gérer les connexions simultanées des joueurs, la création et la gestion des parties, ainsi que l'échange des données nécessaires au bon déroulement du jeu.

Chapitre 2

Définition du sujet et problématique

2.1 Problématique du jeu en réseau

La conception d'un jeu multijoueur en temps réel implique plusieurs contraintes techniques et architecturales. La contrainte majeure concerne la synchronisation des états du jeu entre les différents clients. Chaque joueur doit avoir une vision cohérente de l'ensemble de la partie, incluant les positions de Pac-Man, des fantômes et l'état des pac-gommes. Pour garantir cette cohérence, le projet a opté pour un modèle de serveur autoritaire : toutes les décisions critiques sont validées côté serveur, qui diffuse ensuite les informations mises à jour à tous les clients. Les clients, de leur côté, envoient uniquement leurs intentions de déplacement, qui sont ensuite traitées et synchronisées par le serveur. Les données transitent via des buffers et des queues avant d'être appliquées, assurant un traitement ordonné et fiable des événements du jeu.

2.2 Choix techniques

Le projet a été développé en C++ en utilisant la bibliothèque *Gamedev Framework* (gf), adaptée au développement de jeux vidéo et compatible avec la gestion de l'architecture réseau. La communication réseau repose exclusivement sur le protocole TCP, assurant la fiabilité des échanges entre le serveur et les clients. Les messages échangés sont sérialisés afin de structurer et standardiser les informations transmises.

Le serveur a été conçu selon un modèle multithreadé ou chaîne de responsabilité, reflétant la logique de fonctionnement du jeu : un thread principal gère le serveur et le lobby, tandis que d'autres threads s'occupent des rooms, de la boucle de jeu, des fantômes contrôlés par l'IA et de la communication avec les clients. Cette architecture permet de traiter simultanément les interactions multiples des joueurs et d'assurer une exécution fluide et cohérente des parties.

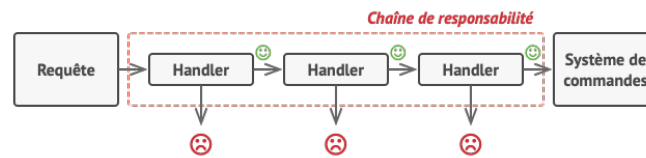


FIGURE 2.1 – Model de Chaîne de responsabilité

2.3 Répartition des rôles

La réalisation de ce projet a été organisée de manière collaborative entre les trois membres du groupe.

Thomas Coutant a principalement travaillé sur le développement du serveur, ainsi que sur la conception de la logique du jeu. Il a également développé un prototype minimal de démarrage, permettant de tester la communication client-serveur avec un simple cube représentant un joueur en mouvement.

Auriane Peter s'est concentrée sur le développement du client, en implémentant l'interface et la gestion des interactions côté joueur.

Benoît Litampha a assuré la communication entre le serveur et le client, participant également à certaines tâches sur le serveur et le client lorsque nécessaire.

Cette répartition a permis une progression simultanée sur plusieurs fronts du projet, tout en garantissant une cohérence globale de l'application.

Chapitre 3

Architecture et conception

3.1 Architecture générale

Description de l'architecture client-serveur, diagrammes éventuels.

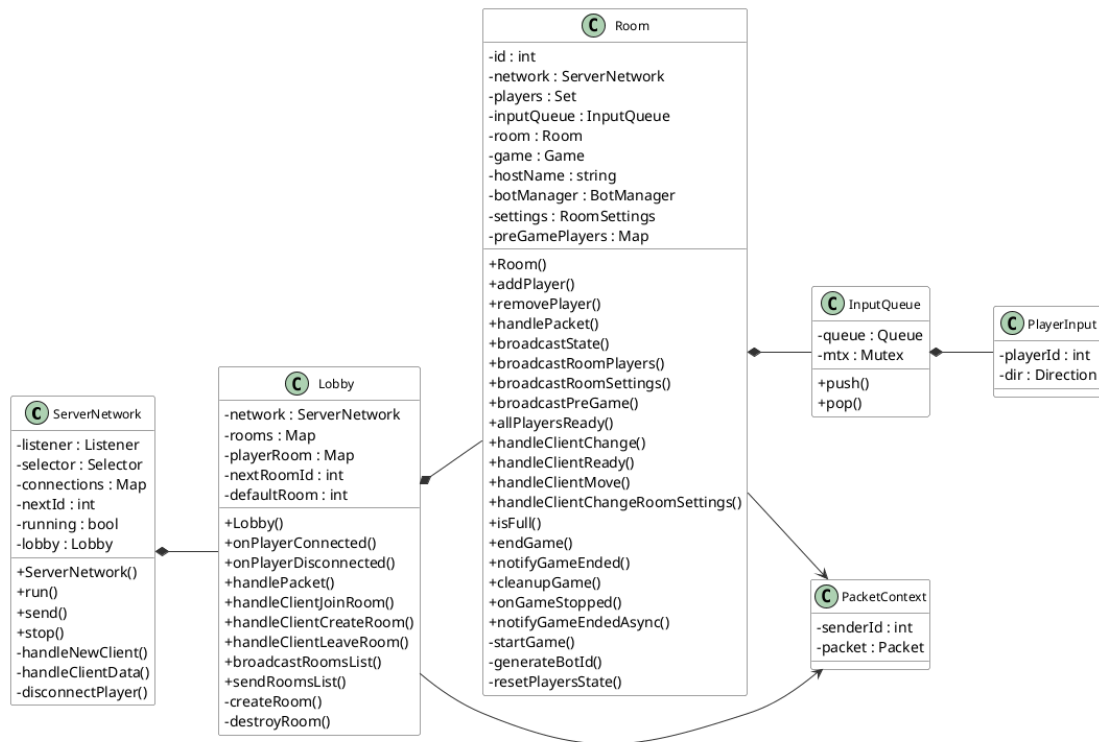


FIGURE 3.1 – Diagrammes de classe de l'architecture réseau

Vue côté serveur - Gestion des packets, des parties, des salles (sans geter et seter)

3.2 Structure du serveur

Gestion des joueurs, des parties, des salles (lobby), synchronisation de l'état du jeu.

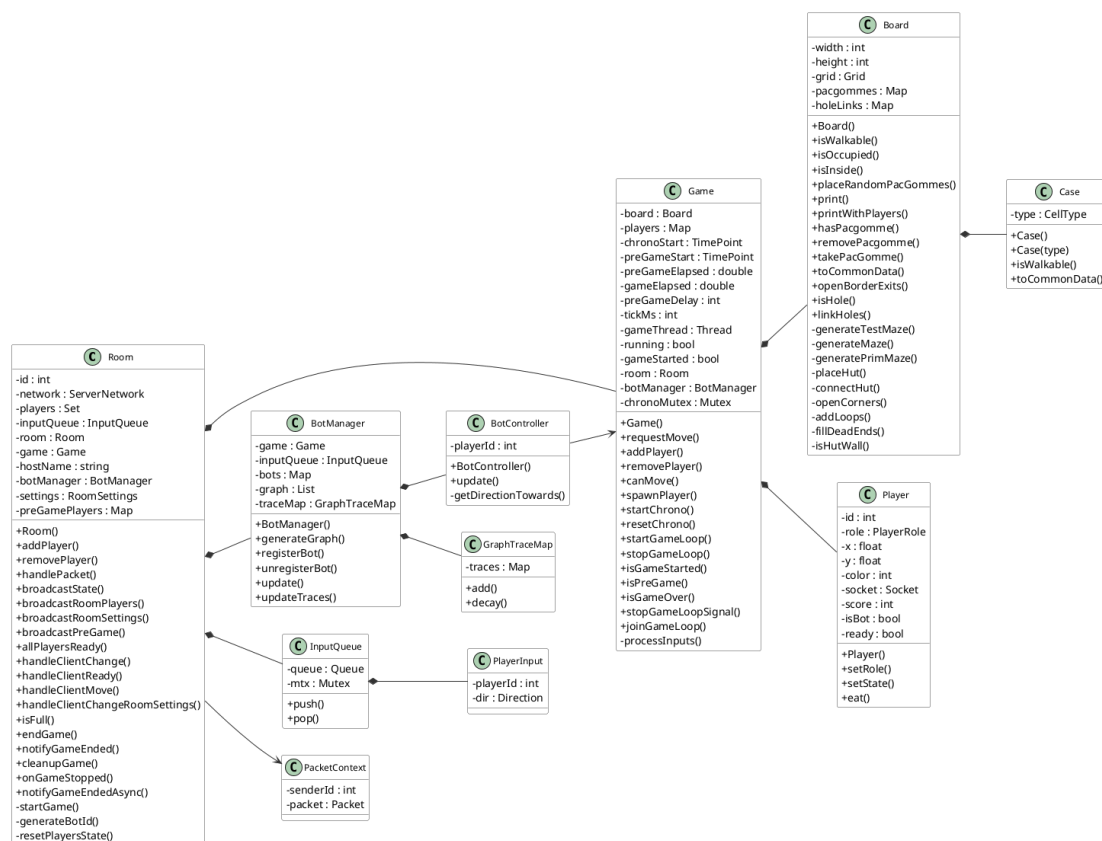


FIGURE 3.2 – Diagramme de classes de l'architecture de jeu

Vue côté serveur - synchronisation, rooms et logique de jeu (sans geter et seter)

3.3 Structure du client

Affichage, gestion des entrées utilisateur, réception et traitement des messages réseau.

3.4 Protocoles et échanges réseau

Description des trames, types de messages, gestion des événements (connexion, mouvement, fin de partie).

Utilisation de `gf : :TCPSocket`, permettant de faire une transmission de données fiable Socket de connexion + communication coté serveur, communication uniquement pour le client Un thread coté client et serveur pour empiler les packet reçu, puis dépilement dans la boucle de jeu pour être traité

Structures communes utilisé par le client et serveur pour la communication (BoardCommonn, PlayerData, RoomSettings, etc...)

Sérialisation/Désérialisation avec `gf : :Packet.is()` et `gf : :Packet.as()` et la surcharge de `|` avec la classe Archive. Déjà implémenté pour les types de base par `gf`

Les structures ont des `gf : :Id`, pour pouvoir les identifier lors de la désérialisation ; Description des strucutres Exemple avec board : le serveur convertit le plateau en BoardCommon, puis le sérialise avec `gf : :Packet.is()`, avant de l'envoyer au client. Le

client traite le packet en regardant si l'id correspond à la structure BoardCommon, puis déserialise avec `gf : Packet.as()`, avant de traiter les données

Pour chaque modification on retransmet toutes les données

Chapitre 4

Réalisation et déroulement du projet

4.1 Mise en place de l'environnement de développement

Prototype client-serveur initial

Avant d'implémenter l'ensemble des mécaniques du jeu, une phase de prototypage a été réalisée afin de tester les choix architecturaux initiaux. L'objectif n'était pas encore de développer un gameplay complet, mais de sécuriser les bases réseau et la synchronisation entre plusieurs clients.

Le prototype consistait en une application minimaliste dans laquelle un client contrôlait le déplacement d'un cube sur un plan 2D. Chaque action utilisateur était transmise au serveur sous forme d'intention de déplacement. Le serveur recevait cette requête, la validait, mettait à jour l'état interne, puis diffusait la nouvelle position à l'ensemble des clients connectés.

Ce modèle repose sur une architecture dite *serveur autoritaire*. Dans cette approche, le serveur est l'unique source de vérité : les clients ne modifient jamais directement l'état du jeu, mais se contentent d'envoyer leurs intentions et d'afficher l'état validé par le serveur.

Ce choix architectural permet de garantir la cohérence globale du système, d'éviter les désynchronisations et de poser des bases solides pour l'extension future vers un véritable jeu multijoueur structuré en rooms et en parties indépendantes.

Ce prototype a ainsi servi de socle technique pour la suite du développement, notamment pour la mise en place du routage des paquets, de la gestion multithread et de la boucle de jeu.



FIGURE 4.1 – Premier prototype

Vue de l'un des clients, il y a 2 client de connecté au serveur

4.2 Conception de l'architecture du serveur

Après validation du prototype initial, l'architecture du serveur a été repensée afin de supporter une structure plus complexe adaptée à un jeu multijoueur complet.

L'objectif principal était de passer d'un prototype monolithique à une architecture modulaire, évolutive et maintenable.

4.2.1 Mise en place de la structure

Architecture multithread

Afin de supporter plusieurs connexions simultanées sans dégrader les performances, le serveur repose sur une architecture multithread. La gestion réseau est détachée de la logique de jeu afin d'éviter qu'une opération bloquante (attente de paquet, latence réseau, déconnexion) ne ralentisse l'ensemble du système.

Un thread est dédié à la réception des paquets. Les messages reçus sont placés dans une file d'attente interne, puis traités de manière synchronisée par la logique centrale du serveur. Cette séparation permet de distinguer clairement la couche communication de la couche métier.

L'utilisation de files de messages agit comme un tampon entre le réseau et la boucle de jeu. Ainsi, le serveur peut continuer à exécuter ses mises à jour périodiques

indépendamment des délais de transmission.

Cette architecture améliore la robustesse du système, limite les risques de blocage et constitue une base adaptée à l'exécution simultanée de plusieurs parties.

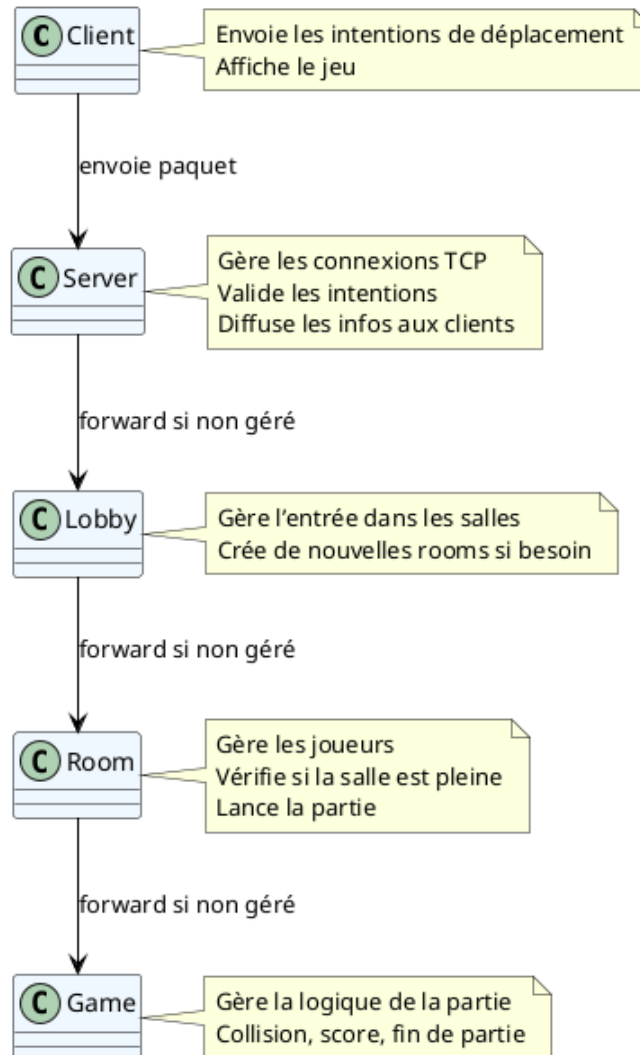


FIGURE 4.2 – Diagramme de Chaîne de responsabilité

Transformation du prototype

Le prototype initial validait la communication réseau et le modèle autoritaire, mais son architecture restait monolithique. L'ensemble des responsabilités, gestion des connexions, organisation des joueurs et logique de jeu, étaient fortement couplées. Cette structure suffisait pour une démonstration technique, mais devenait rapidement limitée dès lors que l'on souhaitait introduire plusieurs parties simultanées.

La transformation du prototype a consisté à structurer le serveur en plusieurs couches distinctes, chacune possédant un rôle clairement défini. La gestion réseau a d'abord été isolée de la logique métier afin de rendre le système plus modulaire. Un système de *Lobby* a ensuite été introduit pour centraliser la création et l'organisation des parties. Chaque partie est encapsulée dans une entité *Room*, responsable de la gestion

des joueurs et de leur état avant le démarrage. Enfin, la logique complète du jeu est contenue dans une classe dédiée *Game*, indépendante des mécanismes réseau.

L'architecture finale suit ainsi une chaîne de responsabilité structurée :

Client → Server → Lobby → Room → Game

Chaque couche agit comme un filtre spécialisé qui délègue la responsabilité au niveau inférieur. Cette hiérarchisation améliore la lisibilité du code, facilite les tests unitaires et permet d'étendre le système (nouvelles règles, nouveaux modes de jeu) sans modifier la couche réseau.

Première implémentation du plateau

La première version du plateau reposait sur une grille bidimensionnelle composée de cases murales et de cases traversables. Cette représentation volontairement simple avait pour objectif de stabiliser le modèle du jeu avant d'introduire des mécanismes plus complexes.

Cette phase intermédiaire a permis de valider plusieurs éléments comme la détection des collisions, la synchronisation des déplacements entre clients et serveur, ainsi que la cohérence entre la représentation logique du plateau côté serveur et son affichage graphique côté client.

Le plateau est ainsi devenu une structure centrale du projet. Il ne constituait plus seulement un support visuel, mais une entité logique partagée servant de référence unique pour les calculs de déplacement, la gestion des interactions et, par la suite, pour la génération procédurale du labyrinthe et l'intelligence artificielle des entités ennemies.

4.2.2 Mise en place du jeu

Boucle de jeu

La boucle de jeu constitue le cœur de la logique serveur. Contrairement à une architecture purement événementielle, le jeu repose sur un modèle à mise à jour périodique (tick serveur).

Le serveur exécute une boucle principale à intervalle régulier permettant :

- De traiter les intentions reçues des clients.
- De mettre à jour les positions des entités.
- De vérifier les collisions.
- De gérer les états temporaires (mode chasseur).
- De mettre à jour le score.
- De vérifier les conditions de fin de partie.

Cette approche garantit que toutes les mises à jour du monde de jeu sont centralisées et synchronisées.

Rôles des joueurs

Au début de chaque partie, les rôles sont attribués :

- Un joueur incarne Pac-Man.
- Les autres incarnent des fantômes.

Chaque rôle possède des règles de déplacement et de vision spécifiques.

Gestion des pac-gommes

Le plateau contient deux types de pac-gommes :

- Pac-gommes classiques.
- Pac-gommes spéciales.

Lorsqu'une pac-gomme classique est consommée :

- Le score de Pac-Man augmente.
- La case devient vide.

Lorsqu'une pac-gomme spéciale est consommée :

- Pac-Man entre en mode chasseur.
- Un timer spécifique est activé.

Gestion du timer

Deux systèmes temporels sont utilisés :

- Un timer global de partie.
- Un timer temporaire pour le mode chasseur.

Le timer global définit la durée maximale d'une partie. Si celui-ci atteint zéro, la condition de victoire est évaluée.

Le timer du mode chasseur est décrémenté à chaque tick. Lorsque celui-ci expire, Pac-Man redevient vulnérable.

Gestion des collisions

Les collisions sont vérifiées à chaque mise à jour :

- Collision Pac-Man / mur → déplacement annulé.
- Collision Pac-Man / fantôme :
 - Si mode normal → perte d'une vie.
 - Si mode chasseur → fantôme éliminé temporairement.

Les collisions sont gérées côté serveur afin d'éviter toute triche ou désynchronisation.

Synchronisation

À la fin de chaque tick, l'état mis à jour est diffusé à tous les clients via un mécanisme de broadcast.

Ainsi :

- Les clients ne calculent jamais la logique.
- Ils se contentent d'afficher l'état reçu.

Plateau et génération procédurale

Le plateau du jeu repose sur une génération procédurale de labyrinthe. L'algorithme utilisé est une variante de l'algorithme de Prim appliquée à la génération de labyrinthes.

L'objectif est d'obtenir :

- Un labyrinthe connexe.
- Une structure cohérente pour le gameplay.
- Une forte rejouabilité grâce à la génération aléatoire.

Algorithme de génération (variante de Prim) Le principe est le suivant :

1. Initialisation complète du plateau en murs.
2. Sélection d'une case initiale (ici $(1, 1)$).
3. Marquage de cette case comme *Floor*.
4. Ajout des murs voisins dans une liste appelée *frontier*.

Tant que la liste *frontier* n'est pas vide :

1. Sélection aléatoire d'un mur dans la *frontier*.
2. Vérification s'il possède au moins un voisin déjà marqué comme *Floor*.
3. Si c'est le cas, suppression du mur entre les deux cases.
4. Marquage de cette case comme *Floor*.
5. Ajout de ses voisins encore murs à la *frontier*.

Ce procédé garantit la création d'un labyrinthe connexe sans îlots isolés.

Ajout de la cabane centrale

Une zone centrale 3×3 , appelée “cabane”, est insérée au centre du plateau.

Cette zone est protégée durant la génération grâce à une fonction empêchant l'algorithme de Prim de modifier ses murs.

Une fonction *connectHut()* garantit qu'au moins une ouverture relie la cabane au reste du labyrinthe, assurant ainsi son accessibilité.

Accessibilité des coins

Les coins du plateau sont destinés aux positions initiales des joueurs. Après génération du labyrinthe, une vérification est effectuée afin de s'assurer que ces zones sont accessibles.

Si nécessaire, des murs adjacents sont ouverts pour relier ces zones au reste du plateau.

Ajout de cycles

L'algorithme de Prim produit un arbre couvrant, donc un labyrinthe sans cycles.

Afin d'améliorer le gameplay et d'éviter des trajectoires trop prévisibles, des ouvertures supplémentaires sont ajoutées aléatoirement après la génération principale.

Cela permet :

- De créer des chemins alternatifs.
- D'éviter un comportement trop déterministe.
- D'enrichir les stratégies de poursuite.

Suppression des culs-de-sac

Les culs-de-sac sont identifiés en détectant les cases ayant un seul voisin accessible.

Certains sont supprimés en cassant aléatoirement un mur adjacent. Cela réduit les pièges trop défavorables pour Pac-Man et améliore la fluidité du jeu.

Portails

Des ouvertures spécifiques sont placées sur les bords du plateau afin de créer des portails. Ces passages permettent à une entité sortant d'un côté du plateau de réapparaître de l'autre côté.

Ce mécanisme introduit :

- Des stratégies d'évasion supplémentaires.
- Une dynamique de poursuite plus complexe.

Intégration gameplay

Le plateau devient ainsi une structure centrale du système :

- Support de la logique de collision.
- Support du pathfinding des fantômes.
- Support du placement des pac-gommes.

La génération procédurale contribue fortement à la rejouabilité et à la variabilité stratégique des parties.

Intelligence artificielle des fantômes

L'intelligence artificielle des fantômes repose sur une architecture hybride combinant :

- Pathfinding classique
- Système d'influence inspiré des colonies de fourmis
- Prise de décision hiérarchique

L'objectif est d'obtenir un comportement :

- Réactif face aux situations critiques
- Cohérent dans ses déplacements
- Émergent lorsqu'il y a plusieurs fantômes

Architecture décisionnelle hiérarchique Le système fonctionne par niveaux de priorité. À chaque tick serveur, le fantôme évalue sa situation selon l'ordre suivant :

Priorité 1 — Fuite stratégique Si Pac-Man est en mode chasseur, le fantôme cherche à rejoindre la cabane centrale.

Pour cela, un algorithme de **Breadth First Search (BFS)** est utilisé afin d'obtenir le plus court chemin non pondéré.

Le BFS garantit :

- Un chemin optimal
- Une complexité maîtrisée
- Une robustesse face aux labyrinthes générés procéduralement

Priorité 2 — Poursuite ciblée Si Pac-Man est détecté dans un rayon de vision défini :

- Un BFS à profondeur limitée est exécuté.
- Le voisin rapprochant le plus rapidement de la cible est sélectionné.

Cette limitation permet :

- De simuler une vision partielle.
- De réduire le coût computationnel.

Priorité 3 — Comportement autonome En absence de menace ou de cible visible, le fantôme adopte un comportement autonome basé sur un système d'influence.

Système d'influence Chaque case du plateau possède un score calculé dynamiquement :

$$Score = Attraction - Répulsions$$

Les composantes sont les suivantes :

- Traces de Pac-Man → Attraction
- Traces de fantômes alliés → Répulsion forte
- Traces de fantômes adverses → Répulsion modérée

Le fantôme sélectionne le voisin présentant le score maximal. Il s'agit d'une approche locale de type *greedy search*.

Comportement émergent Ce système permet l'émergence de comportements collectifs naturels :

- Dispersion automatique des fantômes
- Réduction des regroupements inutiles
- Convergence progressive vers Pac-Man

Sans coordination explicite entre les agents, un équilibre dynamique apparaît.

Techniques utilisées L'implémentation repose sur plusieurs techniques algorithmiques :

- **Breadth First Search (BFS)** — plus court chemin
- **BFS limité** — simulation de vision
- **Recherche gloutonne locale** — sélection optimale immédiate
- **Distance euclidienne au carré** — heuristique rapide
- **Bruit aléatoire contrôlé** — prévention des cycles et oscillations

L'approche hybride permet d'obtenir un compromis entre précision stratégique et coût computationnel maîtrisé.

Rooms : gestion des parties

Afin de supporter plusieurs parties simultanées, le serveur utilise un système de **rooms**. Chaque room est un espace indépendant qui gère :

- La liste des joueurs connectés
- Les rôles attribués à chaque joueur (Pac-Man ou fantôme)
- Les paramètres de la partie (durée, nombre de vies, nombre de pac-gommes spéciales)
- L'état de la partie (en attente, en cours, terminée)

Création et attribution des rôles

- Lorsqu'un joueur rejoint un lobby, il peut entrer dans une room disponible.
- Si aucune room n'est disponible, une nouvelle room est créée automatiquement.
- Les rôles sont assignés en fonction des règles :
 - Un joueur est choisi comme Pac-Man
 - Les autres sont des fantômes

Transmission des informations Chaque room maintient un état centralisé, qui est diffusé à tous les joueurs connectés via le serveur :

- Position des joueurs
- État des pac-gommes
- État des timers (partie et mode chasseur)

Cette approche garantit :

- La cohérence des parties multijoueurs
- L'isolation des parties entre elles
- Une architecture modulaire et scalable

Paramètres de la partie Les rooms permettent de configurer :

- La durée de la partie
- Le nombre de vies de Pac-Man
- Le nombre et la disposition des pac-gommes spéciales

Ainsi, chaque partie peut avoir des règles légèrement différentes, offrant plus de rejouabilité et de flexibilité pour les tests et la compétition entre joueurs.

Règles du jeu

Le jeu respecte des règles spécifiques pour Pac-Man et les fantômes, afin de garantir un gameplay équilibré et stratégique.

Conditions de victoire

- **Pour Pac-Man :**
 - Avoir consommé toutes les pac-gommes du plateau
 - Ou survivre jusqu'à la fin du timer de la partie
- **Pour les fantômes :**
 - Éliminer Pac-Man en lui faisant perdre toutes ses vies

Pac-gommes spéciales Certaines pac-gommes permettent à Pac-Man d'entrer temporairement en *mode chasseur* :

- Pendant cette période, Pac-Man peut éliminer les fantômes.
- La durée du mode est limitée par un timer spécifique.
- Les fantômes affectés réapparaissent après un certain délai.

Vision des joueurs

- Les fantômes ne voient qu'autour d'eux (vision locale limitée).
- Pac-Man voit l'ensemble du plateau.

Cette asymétrie introduit une dynamique de jeu intéressante et stratégique :

- Les fantômes doivent coopérer pour localiser Pac-Man.
- Pac-Man peut planifier ses déplacements en connaissance de cause.

Résumé du système La combinaison de ces règles avec la génération procédurale et l'IA des fantômes permet :

- Un gameplay varié et imprévisible.
- La possibilité pour Pac-Man et les fantômes de développer des stratégies différentes.
- Une forte rejouabilité du jeu.

Chapitre 5

Bilan et perspectives

5.1 Compétences acquises

Programmation réseau, architecture logicielle, travail de conception.

5.2 Résultat final

Fonctionnalités implémentées, objectifs atteints ou partiellement atteints.

5.3 Améliorations possibles

idee.txt

Conclusion

Ce projet a permis de mettre en pratique les connaissances acquises durant la formation en informatique, en particulier dans le domaine du développement réseau et de la conception logicielle.

Sitographie

Résumé

Mots-clés

C++ - PacMan - GF - Réseau - [AUTRE]

Abstract

Key-Words

MathLive - JavaScript - TypeScript - HTML - Drag & Drop